

SIH 2026 — FULL PROJECT DOCUMENTATION

Team Technical Understanding Document

1. Project Overview

The platform is designed for trusted identity and lifecycle management of physical and digital assets. Its purpose is to create a reliable digital representation of an asset, maintain important lifecycle/provenance information through blockchain-backed records, associate actions with authorized identities, and provide fast application-level access through an indexed SQL database.

The architecture is hybrid rather than purely blockchain-based. Blockchain is used where immutability, provenance, authorization, and trust are important. Conventional backend and database technologies are used where application performance, querying, and usability are important.

Core architectural principle: blockchain provides the authoritative trust layer, while SQL provides a fast derived/queryable representation of the blockchain state and events.

2. High-Level Architecture

Frontend: React + Vite + Ethers.js + MetaMask

Backend: FastAPI + Python + SQLAlchemy Async

Identity: SIWE / EIP-4361 + did:ethr + short-lived JWT

Blockchain: Ethereum-compatible network + Solidity ERC-721 smart contract + OpenZeppelin AccessControl

Indexing: Python background Event Indexer + Web3.py

Database: SQLite through SQLAlchemy

Verification: Public `/verify/:serial` route

Development: Hardhat + local Ethereum-compatible node, Chain ID 31337

3. Complete System Flow

Identity flow:

Wallet → SIWE message/nonce → wallet signs message → FastAPI verifies signature → wallet address recovered → DID representation → short-lived JWT → authenticated application session.

Asset lifecycle flow:

Authorized manager → React UI → backend/blockchain interaction → smart contract checks `MANAGER_ROLE` → asset lifecycle operation → blockchain state/event → Event Indexer → SQL database → dashboard.

Verification flow:

Verifier → `/verify/:serial` → application retrieves verification information → asset/provenance information is presented without requiring platform login.

Indexing flow:

Smart Contract → Blockchain Event → Web3.py Event Indexer → SQLAlchemy → SQLite → React dashboard.

4. Blockchain Layer

4.1 Ethereum-Compatible Network

The prototype runs against a local Ethereum-compatible development network. The Chain ID used is **31337**. This is a development configuration rather than a production deployment.

4.2 Solidity

The smart contract is written in **Solidity v0.8.24**. The contract contains the on-chain rules governing asset representation and restricted lifecycle operations.

4.3 ERC-721

Assets are represented as ERC-721 tokens. ERC-721 is appropriate because each asset is individually identifiable and non-fungible. Each token can have its own identity and lifecycle rather than representing interchangeable units.

4.4 OpenZeppelin AccessControl

The contract uses OpenZeppelin's AccessControl mechanism. The important application role is **MANAGER_ROLE**. Operations that require management authority are restricted at the smart-contract level.

4.5 On-Chain Events

The contract emits lifecycle-related events including **Transfer**, **Mint**, and **Revoke**. These events form the input for the off-chain event indexing system.

4.6 Hardhat

Hardhat is used as the development environment for compiling, testing/deploying, and working with the Solidity contract.

5. Why the Smart Contract Matters

The smart contract is not simply another database table. It is the blockchain-resident program that defines rules and can enforce permissions independently of the frontend.

For example, hiding a mint button in React would only be a user-interface restriction. Using **MANAGER_ROLE** in the smart contract makes authorization an on-chain rule. A caller without the required role should not be able to perform the protected operation merely by bypassing the UI.

6. Identity Architecture

6.1 SIWE — Sign-In With Ethereum

SIWE is used to authenticate users through ownership of an Ethereum-compatible wallet. The backend generates a cryptographic nonce and authentication message. The user signs the message through the wallet. The backend then verifies the signature and recovers the wallet address.

6.2 EIP-4361

The SIWE flow follows the Sign-In with Ethereum message standard associated with EIP-4361. The signed message provides structured information for the authentication process.

6.3 Nonce

A server-generated nonce makes the authentication request fresh. Its purpose includes preventing an old signed authentication message from simply being replayed as a new login.

6.4 Non-Custodial Authentication

The application does not need the user's private key. The wallet performs the signing operation. The backend receives the signed message/signature and verifies it.

6.5 Signature Verification

The backend uses **eth_account.Account.recover_message** to recover the Ethereum address corresponding to the signed message and compare it with the identity being authenticated.

7. Decentralized Identity — did:ethr

The prototype uses **did:ethr** as its decentralized identity representation. It associates the identity with an Ethereum-based identifier and provides a standardized DID-oriented representation.

In the architecture, SIWE establishes control of the wallet, while did:ethr provides the identity representation used by the platform. This supports the project's interoperability objective with decentralized identity systems.

8. JWT Session Layer

After wallet-based authentication succeeds, the backend creates a short-lived JWT. The prototype uses a **30-minute** JWT lifetime.

This separates two concepts:

- SIWE/signature: establishes proof of wallet ownership.
- JWT: maintains the authenticated application session.

Without a session token, the user could be required to repeatedly perform wallet signing for ordinary API operations. JWT makes subsequent application requests more practical while keeping the session short-lived.

9. Backend Architecture

9.1 FastAPI

FastAPI is the Python backend framework. It exposes the application's API endpoints and supports asynchronous operations.

9.2 SQLAlchemy Async

SQLAlchemy is used as the database toolkit/ORM. The prototype uses its asynchronous capabilities for database operations.

9.3 SQLite

SQLite is the prototype SQL database. It is lightweight and convenient for development and demonstration.

9.4 Web3.py

Web3.py provides the Python backend with communication capabilities for the Ethereum-compatible blockchain. It is also used by the event indexing component.

9.5 python-jose

python-jose is used for JWT handling in the backend.

9.6 Database Initialization

At startup, the backend dynamically initializes the required SQL tables using **Base.metadata.create_all**.

10. Event Indexer

The Event Indexer is the central bridge between the blockchain trust layer and the normal application database.

Smart Contract → Event → Python Indexer → SQL

The Python backend runs a background asynchronous task that monitors blockchain events through Web3.py. When a relevant event is detected, the indexer transforms/synchronizes the information into the SQL database.

This produces a queryable off-chain representation without changing the blockchain's role as the authoritative source.

10.1 Why an Indexer Exists

- Blockchain data is not designed to replace every type of application database query.
- Complex lists, filtering, joins, and dashboard queries are more naturally handled through SQL.
- Repeatedly querying blockchain state for every frontend operation can be slower and less convenient.
- The indexer allows the application to use SQL for normal reads while retaining blockchain-backed provenance.

10.2 Source-of-Truth Relationship

Blockchain = authoritative source.

SQL = derived/indexed representation.

The SQL database should therefore be treated as a synchronized application index rather than the final authority over the asset's blockchain history.

11. Frontend Architecture

11.1 React + Vite

The user interface is implemented using React with Vite as the frontend development/build environment.

11.2 Ethers.js

Ethers.js provides the frontend with Ethereum-compatible blockchain and wallet interaction capabilities.

11.3 MetaMask

MetaMask is used as the user's wallet interface. It handles wallet ownership, signing, and blockchain transaction interaction without exposing the private key to the application backend.

11.4 wallet_watchAsset / EIP-747

The frontend uses the wallet_watchAsset capability associated with EIP-747 to request that the asset be displayed/imported in the user's MetaMask wallet.

11.5 localStorage Deduplication

The frontend stores local flags using keys such as **mm_imported_** to remember that an asset has already been offered/imported. This prevents repeated wallet import prompts during subsequent dashboard loads.

12. Public Verification

The dedicated **/verify/:serial** route provides a public verification interface. It does not require the verifier to log in.

The intended flow is:

- A verifier receives an asset serial number.
- The verifier opens the public verification route.
- The application locates the relevant asset/provenance information.
- The verifier can inspect the system's recorded information without needing a platform account.

This is useful for external parties such as customers, auditors, logistics partners, or other organizations that need to verify an asset record.

13. Physical and Digital Assets

The architecture is designed to represent both physical and digital assets through digital asset records/tokens and their associated lifecycle information.

Important conceptual boundary: blockchain establishes the integrity and provenance of the digital record. It does not magically establish that a physical object is genuine.

For a physical asset, the real-world authenticity depends on the registration process and the evidence used to associate the physical object with its digital representation. Blockchain then provides a tamper-evident history of that registered digital representation.

14. Asset Lifecycle

The platform treats an asset as having a lifecycle rather than merely being a static record.

- Registration/minting creates the digital asset representation.

- Ownership and transfer information can be represented through ERC-721 mechanisms and Transfer events.
- Authorized lifecycle operations can update the asset state.
- Revocation provides a way to mark an asset as no longer valid/active within the contract's lifecycle model.
- Events provide a historical trail that the indexer can synchronize into SQL.

15. Security Model

15.1 Wallet-Based Authentication

Users authenticate by demonstrating control of a wallet through a signed SIWE message rather than submitting a traditional password to the application.

15.2 Replay Protection

The nonce makes the authentication message specific to a login attempt and reduces the risk of replaying a previously captured signature.

15.3 On-Chain Authorization

MANAGER_ROLE is enforced in the smart contract for protected lifecycle operations.

15.4 Short-Lived Sessions

JWT sessions expire after the configured short lifetime of 30 minutes in the prototype.

15.5 Active-User Validation

The authentication middleware checks **User.is_active**. This prevents an inactive/deleted application user from continuing to authenticate simply because they previously had a valid wallet session/signature.

16. Technical Problems Solved

16.1 MetaMask Import UX

A practical usability issue was preventing the wallet from repeatedly prompting users to import the same asset. The prototype automatically handles wallet asset import when appropriate and uses localStorage-based deduplication to avoid repeated prompts.

16.2 Orphaned Blockchain Assets

A database user can be deleted while blockchain assets associated with that wallet can still exist on-chain. Deleting the SQL user does not delete the blockchain asset.

The prototype therefore performs a pre-deletion check using the on-chain **getAssetsByOwner** functionality and identifies assets that should be revoked/handled before the corresponding database entry is purged.

16.3 Authentication After User Deletion

An old authentication state could otherwise survive the deletion of a user record. The prototype addresses this through strict **User.is_active** checking in authentication middleware so inactive users are rejected.

17. Data Responsibilities

Layer	Primary responsibility
Smart contract	Authoritative asset state, protected lifecycle rules, role authorization, blockchain events
Blockchain	Persistent/tamper-evident record of on-chain state and events
Event Indexer	Reads blockchain events and synchronizes them into SQL
SQL database	Fast application-level queries and indexed representation
FastAPI	Application APIs, authentication/session handling, backend orchestration
React	User interface and dashboard/verification presentation

18. Why the Hybrid Architecture

A blockchain-only application would make every application concern dependent on blockchain access patterns. That is not ideal for fast dashboards, complex queries, or ordinary web application workflows.

A database-only application would be efficient but would not provide the same blockchain-backed tamper-evident trust model for the selected lifecycle records.

The hybrid architecture therefore separates concerns:

- Blockchain handles trust, provenance, authoritative on-chain state, and protected lifecycle operations.
- SQL handles fast reads, application queries, and dashboard performance.
- The Event Indexer keeps the two representations synchronized.

19. End-to-End Technical Sequence

1. A user connects a wallet through MetaMask.
2. The backend provides a server-generated authentication nonce/message.
3. MetaMask signs the SIWE message.
4. FastAPI verifies the signature and recovers the wallet address.
5. The platform represents the identity using did:ethr and creates a short-lived JWT.
6. An authorized manager uses the React dashboard to perform an asset operation.
7. The smart contract checks MANAGER_ROLE where authorization is required.
8. The blockchain records the resulting state/event.
9. The Python Event Indexer detects the event through Web3.py.
10. The indexer updates the SQL representation.
11. The React dashboard reads the indexed data for normal application display.
12. An external verifier can use /verify/:serial without logging in.

20. Development Configuration

Backend language: Python

Frontend language: JavaScript

Smart contract language: Solidity v0.8.24

Contract standard: ERC-721

Access control: OpenZeppelin AccessControl / MANAGER_ROLE

Backend framework: FastAPI

ORM/database toolkit: SQLAlchemy Async

Database: SQLite

Blockchain Python library: Web3.py

Blockchain frontend library: Ethers.js

Wallet: MetaMask

Authentication: SIWE / EIP-4361

Identity: did:ethr

Session: python-jose JWT, 30-minute prototype lifetime

Development framework: Hardhat

Local chain ID: 31337

21. Business / Deployment Concept

The platform is intended as a B2B service rather than only a demonstration application.

- SaaS subscription per user/dashboard.
- Verification API fees, including access patterns represented by endpoints such as `/verify/:serial`.
- Minting/deployment or asset registration fees.
- Value through fraud mitigation and automated audit/compliance workflows.
- Potential markets include logistics and supply chain, luxury/high-value assets, and institutional identity.

22. Production Evolution

The current implementation is a prototype. A production system would need additional infrastructure and security work.

- Move from SQLite to a production database such as PostgreSQL.
- Deploy the smart contract to an appropriate production or supported test network.
- Introduce robust key/role management and operational controls.
- Add production monitoring, logging, alerting, backups, and recovery procedures.
- Harden API authentication, authorization, rate limiting, validation, and secrets management.
- Consider blockchain transaction costs, throughput, confirmation handling, and failure/retry behavior.
- Secure and audit the smart contract before handling real assets or significant value.
- Design indexing recovery/reconciliation so SQL can be rebuilt or repaired from authoritative blockchain data.
- Add scalable deployment infrastructure for the API, indexer, frontend, and database.

23. System Concepts in Plain Language

Term	Meaning in this project
Blockchain	A distributed ledger used here as the trusted on-chain record.
Smart contract	The program on the blockchain that defines and enforces asset rules.
ERC-721	A token standard for unique, individually identifiable assets.
Role-based access control	Permission system that restricts contract operations to assigned roles.
MANAGER_ROLE	The contract role used for authorized management operations.
Event	A blockchain-emitted record that describes an important state/action occurrence and can be consumed by the indexer.
Indexer	A service that reads blockchain events and creates a query-friendly database representation.
SQL	The query language/database layer used for fast application data access.
SIWE	Wallet-based sign-in where the user signs a structured authentication message.
EIP-4361	The standard associated with the Sign-In with Ethereum message format.
Nonce	A fresh authentication value used to prevent replay of old signed messages.
DID	A decentralized identifier representation.
did:ethr	An Ethereum-associated DID method used by the prototype.
JWT	A signed token used to maintain an application session.
Web3.py	Python library for interacting with Ethereum-compatible blockchain systems.
Ethers.js	JavaScript library for frontend Ethereum/wallet interaction.
MetaMask	Wallet software used for signing and blockchain interaction.
EIP-747	Wallet asset-watch/import capability used for better asset display UX.
FastAPI	Python web framework used for backend APIs.

SQLAlchemy	Python database toolkit/ORM used to work with the SQL database.
SQLite	Lightweight SQL database used in the prototype.
Hardhat	Development environment for Ethereum smart-contract development.
Chain ID 31337	Identifier of the local development network used by the prototype.

24. Key Design Decisions

- **ERC-721**: selected because assets are unique.
- **AccessControl**: selected to enforce role-based permissions on-chain.
- **SIWE**: selected to authenticate wallet ownership without collecting private keys/passwords.
- **did:ethr**: selected to provide a standardized Ethereum-linked decentralized identity representation.
- **JWT**: selected to maintain a practical short-lived web session after wallet authentication.
- **Event Indexer**: selected to bridge blockchain authority and application-level query performance.
- **SQLite**: selected for lightweight prototype development.
- **React**: selected for the interactive web dashboard.
- **MetaMask**: selected as the wallet interface for signing and blockchain interaction.
- **Public verification route**: selected so external verifiers do not need platform accounts.

25. The Complete Mental Model

Blockchain → **Trust**

Smart Contract → **Rules**

SIWE / DID → **Identity**

JWT → **Session**

Event Indexer → **Bridge**

SQL → **Speed**

React → **User Interface**

These seven concepts describe the architecture at the highest useful level. Understanding how data moves between these seven layers is the foundation for understanding the entire prototype.

26. Architecture Summary

The project is a hybrid Web3/Web2 asset platform. The blockchain and smart contract provide the authoritative trust and lifecycle layer. ERC-721 gives each unique asset an individually identifiable digital representation. OpenZeppelin AccessControl and MANAGER_ROLE protect restricted lifecycle actions. SIWE lets users authenticate through wallet ownership without giving the application their private keys. did:ethr provides a decentralized identity representation, while JWT provides a short-lived application session.

The FastAPI backend communicates with the blockchain through Web3.py and works with SQL through SQLAlchemy Async. A background Event Indexer listens to blockchain events and synchronizes them into SQLite. The React/Vite frontend uses Ethers.js and MetaMask for wallet interaction and queries the SQL index for fast dashboard operations. wallet_watchAsset improves asset display in MetaMask, with localStorage preventing repeated import prompts. Finally, /verify/:serial provides a public verification path for external parties.

The central architectural idea is therefore: **use blockchain for what requires trust and provenance, and conventional web/database technology for what requires performance and usability.**

27. Project Vocabulary Map

Asset: the physical or digital item represented by the platform.

Token: the blockchain representation of a unique asset through ERC-721.

Owner: the wallet address associated with the token's ownership state.

Manager: an address granted MANAGER_ROLE for protected operations.

Lifecycle: the sequence of registration, ownership/state changes, and possible revocation.

Provenance: the recorded history associated with the asset's digital representation.

On-chain: information/state recorded through the blockchain and smart contract.

Off-chain: application information stored outside the blockchain, such as SQL indexing data.

Index: the query-friendly representation derived from authoritative events.

Verification: checking the recorded digital asset/provenance information through the platform.

End of technical project documentation.